

Structural Testing Framework for RRAM DNN Accelerators: From Circuit to System-Level

Emmanouil Anastasios Serlis¹, Emmanouil Arapidis¹, Anteneh Gebregiorgis¹, Hassen Aziza²,
Mottaqiallah Taouil^{1,3}, Said Hamdioui^{1,3}, Moritz Fieback¹

¹Computer Engineering Laboratory, Delft University of Technology, Mekelweg 4, 2628CD, Delft, The Netherlands

²IM2NP, Aix-Marseille Université, Marseille, France ³CognitiveIC, Van der Burghweg 1, 2628CS, Delft, The Netherlands

Email: {e.a.serlis, s.hamdioui, m.c.r.fieback}@tudelft.nl

Abstract—Analog computation-in-memory (CIM) using Resistive Random Access Memory (RRAM) provides high-performance computations for DNN workloads. Current works regarding RRAM’s unique manufacturing defects either rely on abstract system-level fault modeling or extensive March-type test algorithms that struggle to scale to the full DNN level. To tackle this issue we provide a 3-step system-level structural testing methodology for RRAM AI Accelerators, which covers all 3 levels of a hardware accelerator, namely the circuit-level, the single-layer level and the full DNN one. Starting from the RRAM crossbar-level we propose fault models that can be used to develop effective test patterns. Next, we map these crossbar results upwards to the layer level and finally the complete DNN. Finally, we map the crossbar-level results to actual DNN workloads, for fault criticality estimation and validation of the 3-level fault modeling. We also propose a synthetic test image generation pipeline, for multi-layer fault sensitization, to reduce overall test time. Compared to crossbar-only test patterns, our results showcase a maximum improvement of 20 % with the deployed balanced test pattern generation scheme on crossbar-level testing, as well as a 40 % reduction of the expected image-based test time, with sub-5% defect coverage drop, while still covering all RRAM & ADC unique defects.

Index Terms—Structural Test, Computation-in-memory, RRAMs, Deep Neural Networks

I. INTRODUCTION

CIM AI accelerators are promising alternatives to conventional GPUs and digital chips [1]. While commercialized CIM chips often utilize easily integrated digital SRAM, they suffer from weight volatility and limited efficiency [2]. Analog RRAM-based CIM resolves these issues by enabling scalable, low-power, and low-latency MAC operations [3]. However, RRAM devices exhibit numerous reliability issues and manufacturing defects [4, 5], necessitating comprehensive circuit- and application-level mitigation strategies.

Recent works on reliability & testing of analog RRAM CIM architectures fall into 3 categories; structural, system-level and hybrid. Regarding structural approaches, their characteristic relies solely on circuit-level defect injection, with almost non-existent inclusion of DNN-level analysis [6–8]. An indicative work is that of [9], where Hou et al. offer equivalent fault models for the classical open and short defect models for RRAM-based SNNs. However, their approach completely ignores system-level functional accuracy, risking severe yield loss by rejecting chips that are structurally flawed but still algorithmically capable, implying the need for DNN testing.

Other literature aims at providing application-specific, system-level mitigation techniques [10–12]. The main limitation of these works is the inclusion of simplified fault models in order to focus more on fast DNN-based validation. For instance, Xia et al. addresses RRAM crossbar yield issues by proposing a software-based system-level fault-aware retraining methodology via stuck-at logic fault modeling, which fully ignores RRAM-specific defects. Finally, there is a niche combination of works that combine circuit level and DNN-level analysis [13–15]. However, such approaches generate analytical RRAM data to observe a specific reliability issue, such as conductance drift during DNN inference, and do not extend their analysis to deploy cell-aware [16] or device-aware [17] testing methods for extensive and realistic defect coverage. Tackling these shortcomings requires not only generating extensive measurement or SPICE-based tests which is time-consuming, but also developing analytical fault models to connect crossbar-level to DNN-level test generation effectively.

This paper, to the best of our knowledge, is the first to perform a system-level test for RRAM-based AI Accelerators. We start by initially identifying maximum coverage for all different defect and non-idealities in analog RRAM crossbars via exhaustive RRAM and peripheral circuit-level defect injection. Next, we provide the required fault models on a circuit and system-level, that will allow us to map circuit test results to full DNN scale. The main contributions of the paper are to:

- Define fault models across all levels of a RRAM-based AI Accelerator, with thorough circuit and DNN level validation of them.
- Propose a test pattern generation scheme for crossbar-level defect coverage in a complete system.
- Generate deterministic and gradient-based structural test images for cross-layer controllability, reducing overall RRAM DNN test time.

The remainder of the paper is structured as follows. Section II provides the required background. Section III provides an overview of the proposed methodology. Section IV offers methodology’s application details. Section V provides extensive results of the methodology across all development levels. Section VI offers a comparison of the proposed methodology with RRAM DNN test methodologies found in literature. Section VII concludes the paper.

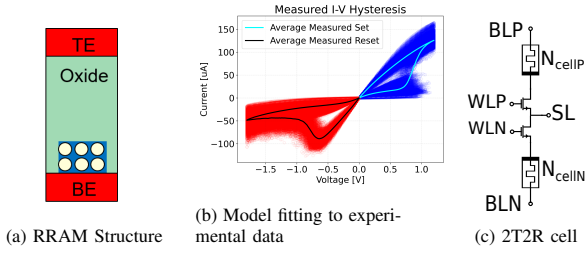


Fig. 1: Overview of RRAM device technology

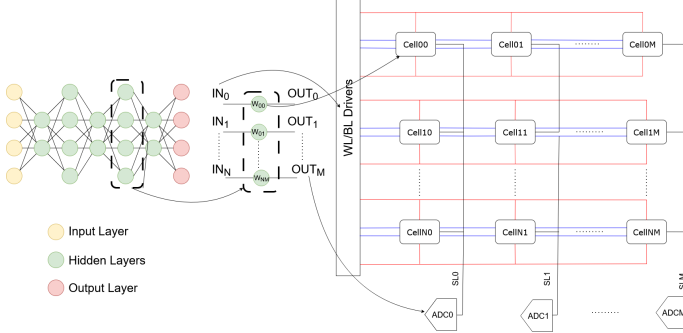


Fig. 2: Mapping of Neural Network to Crossbar of N rows and M columns

II. BACKGROUND

A. RRAM technology

RRAM devices usually follow the structure of Fig. 1a, being a 2-electrode device (TE,BE) for write and read operations. The oxide in-between the 2 electrodes is responsible for storing the RRAM state. In particular, an RRAM device is commonly fabricated at a high-resistance state, where the oxygen ions are disrupting filament creation. During electroforming, as well as during SET memory operations, a positive voltage is applied to TE which drive the oxygen ions away, thus leading to a low-resistance state (LRS). While at LRS, when a positive voltage is applied to BE with a grounded TE, then oxygen ions disrupt CF to return. Indicative RRAM measurement data alongside their fitting curve are shown in Fig. 1b, where the characteristic I-V curve is demonstrated. From the plot, it is obvious that even non-defective measurements can be governed by significant conductance drift, in either LRS or HRS states, thus affecting device operation. Finally, different types of RRAM memory cell designs exist such as 1R, 1T1R and 2T2R [18]. As part of this work, RRAM cell structure is assumed to be solely of 2T2R type as shown in Fig. 1c, since such architecture not only showcases highly competitive performance in fabricated analog RRAM CIM AI chips [19], but also comes with a more complex testing scheme in terms of more potential input operations and defective nodes.

B. Computation-in-Memory

Mapping of DNN workloads to analog RRAM CIM crossbars is visualized in Fig. 2. A conventional DNN includes multiple layers with each layer having its own input-weight-output (IN/W/O) configuration. The inputs of a DNN layer are mapped across multiple parallel crossbar rows, which are provided via the differential BLs and WLs of the RRAM 2T2R

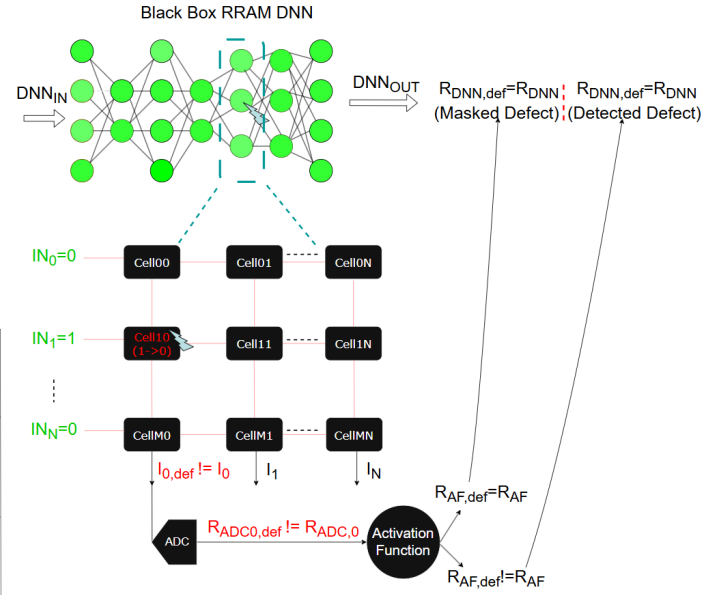


Fig. 3: RRAM DNN Structural Testing Problem

cells. The input could come in binary, ternary or multi-level format. For the later configuration, it is unusual to include voltage-based digital-to-analog converters due to challenges to driving multiple RRAM cell per row. Next, each memory 2T2R module $Cell_{i,j}$ includes the DNN weights in the form of various conductance states. These states - similarly to the INs - could come in binary, ternary or multi-state form depending on the hardware constraints and the software-based quantization. The $IN \times W$ products are then accumulated across columns via a collective output current I_{SL_i} which is then read and digitized via an analog-to-digital converter (ADC).

III. PROPOSED METHODOLOGY

A. Overview

Test development for DNNs can be done on three different levels: full DNN level, single layer level, or on the crossbar level. Each of these layers has several benefits but more importantly also drawbacks when developing a test, resulting in a testing gap as we will discuss next.

1) *Testing gap*: We discuss the testing gap based on Fig. 3. On the full DNN level, tests can be easily applied to the inputs (DNN_{IN}), and the resulting outputs (R_{DNN}) can be observed. When a defect in the DNN ($R_{DNN,def}$) results in a different output compared to the defect-free case ($R_{DNN,def} \neq R_{DNN}$), the defect is detected, otherwise the defect is masked and not detected. However, these tests are global and cannot detect faults in all layers of the network, due to the limited controllability of deeper layers or the limited observability of earlier layers. For example, controlling the marked layer in Fig. 3 requires that the test vector is propagated from input through three hidden layers. The configuration of every layer limits how well these patterns can propagate, and thus limit the number of faults that can be sensitized in the layer. Test development on this level is done in software, such as PyTorch.

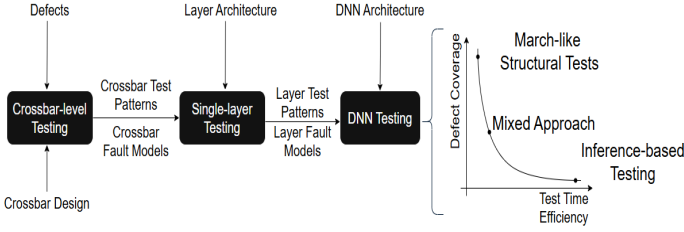


Fig. 4: 3-step methodology for RRAM DNN test development

This allows for quick design iterations of the network, but realistic defect and fault models that relate to the actual hardware cannot be used, thus resulting in low-quality tests.

On the single layer level, the controllability and observability are better, which increases the ability to sensitize faults. However, this comes at the cost of additional hardware overhead to access this layer. Furthermore, some faults will still not be detectable, because the activation function will mask them. In Fig. 3, a defect in Cell10 is detected if the appropriate input pattern (in this case 01...0) is applied and if the output after the activation function (R_{AF}) is different from the defect-free case, i.e., $R_{AF, def} \neq R_{AF}$. If $R_{AF, def} = R_{AF}$, the fault is masked. Test development on this layer is still done in higher-level software frameworks, but allows for a more detailed fault analysis.

On the crossbar level, there is full control over all inputs, while all outputs can be observed directly from the ADC output (R_{ADC}). This gives the best observability, at the highest cost of accessing these circuits. For instance, in Fig. 3 the defective Cell10 in column 0, results in wrong ADC code $R_{ADC,0,def} \neq R_{ADC,0}$. Test development is done at the SPICE level, allowing the usage of accurate defect models. This will result in the most realistic fault models for the particular circuit, and thus in the highest-quality tests. Unfortunately, SPICE simulations are slow and cannot be directly used to develop tests for the complete system.

2) *Proposed Methodology*: We propose a framework that uses low-level defect simulations at the crossbar level to derive accurate crossbar-level fault models and tests for them. Next, we propagate these fault models upwards to the single layer-level and derive layer-level fault models and tests. Finally, we propagate these upward to the full DNN, resulting in accurate and quick tests based on realistic defect models.

After validating the need for hardware-level simulation in order to accurately perform RRAM DNN structural testing, we showcase the proposed testing pipeline in Fig. 4. Overall, we propose a 3-step methodology start from the crossbar-level, and then continues to the single and full DNN ones. We start by defining the crossbar circuit design as well as the targeted crossbar defects. Next, via analyzing all circuit input combinations, we are able to generate descriptive fault models, which encapsulate not only the entire input-defect space, as well as the defined crossbar-level faulty behaviors. This validated fault space leads to maximum-coverage, minimum-delay test patterns.

Next, we define the architecture of each layer, which is

used to propagate the generated crossbar test results to a single-layer level, both in terms of equivalent fault models and in terms of test vectors based on the single-layer input-output integer of floating-point (FP) format. Similarly, the DNN architecture between all single-layer components needs to be known to perform end-to-end DNN testing starting from the generated single-layer fault models and test patterns. DNN structural testing can be done in 3 ways: one where all DNN RRAM layers have to be thoroughly tested via low-level test vectors, with read and write operations available. This approach maximizes defect coverage at the cost of extremely expensive test time, due to the increased delay added by the RRAM write operation. On the other side, an inference-based test method will use only DNN inputs as test operations, saving test time, since only read operations will be used on the crossbar, but also minimizing defect detection. As a tradeoff, a mixed-mode approach, with certain defect types and DNN layers being tested via structural test patterns to achieve a satisfactory defect detection metrics and others via inference-based ones would give a balanced between low test time and still high enough defect accuracy, where at least the highest defect strength values per defect case are going to be detected.

B. Circuit-level Structural Testing

Section III-B lays the foundation of hardware-aware system-level DNN testing. Initially, we define the device and circuit defects and non-idealities that are going to be examined. Next, we propose fault models in the form of compact fault notations, that efficiently summarize all combinations of inputs, outputs as well as faulty behaviors. Finally, we use this notation to develop a test optimization problem for optimal circuit-level test patterns leading to maximum coverage and minimum test cost. This Section's results are then going to be propagated to the DNN scale.

1) *Crossbar Defect Modeling*: The structural testing problem starts from the analog RRAM crossbar array of Fig. 2. We categorize defects into two categories. The first one refers to the resistive defects, such as opens, shorts and bridges, based on the cell-aware test methodology, as shown in Fig. 5. Such defects could affect both CMOS and RRAM devices and subsequently all crossbar sub-circuits, such as WL/BL drivers, RRAM cells, and ADC readout. Next, since RRAM devices come with their own unique manufacturing defects, we also have to deploy defects based on the device-aware method. As a result, maximum defect detection can happen within the RRAM crossbar, whereas subsequent DNN operations and non-defective RRAM layers can only worsen detectability.

2) *Crossbar Fault Modeling*: For regular memory testing, the $\langle S/F/R \rangle$ notation is deployed, where S denotes the cell sensitizing pattern F describes the cell RRAM state after applying S and R is the peripheral circuit readout value [20]. For the analog RRAM CIM macro of Fig. 2, we propose an equivalent notation $FN_{CR} = \langle \langle S_0/F_0 \rangle, \langle S_1/F_1 \rangle \dots \langle S_N/F_N \rangle / R_{ADC} \rangle$, where each of the $\langle S_i/F_i \rangle$, $i \in [0, N]$ symbolize the applied input to the column cell i and the RRAM cell numerical state difference after applying S_i to S_N are the parallel

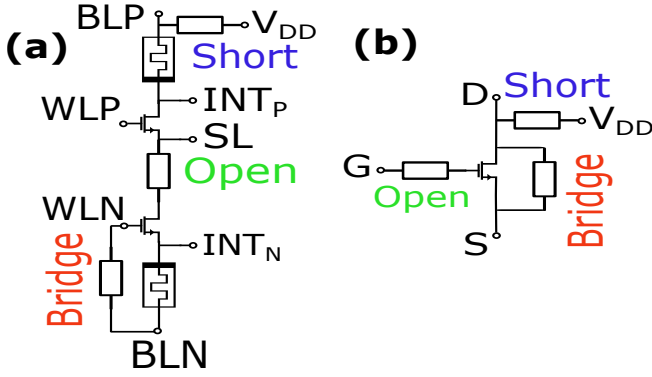


Fig. 5: Cell-level resistive defects for (a) 2T2R and (b) CMOS cells

applied patterns across the RRAM crossbar rows 1 to N . All contributions of the N cells of a common RRAM crossbar column, including the 1 defective cell and the $N-1$ non-defective ones are being read by a single ADC, with R_{ADC} showcasing the ADC readout difference. As a result, RRAM state values F_i of 0 indicate that operation S_i hasn't lead to any change in the RRAM cell state, and similarly $R_{ADC} = 0$ indicates that the defective cell contribution across the RRAM column isn't enough to cause readout defect detection.

3) *Crossbar Test Development*: Based on the modified FN_{CR} notation, we can generate all input-defect combinations in the form of a binary matrix, where '1' elements indicate ADC-based defect detection and '0' indicates no detection. For efficient defect-oriented testing, based on the aforementioned detectability matrix, we want to generate a test vector which is going to provide maximum defect coverage for all crossbar-level examined defects, while minimizing equivalent test time. Test time is minimized via selecting a test vector which has a minimum amount of inference-based elements and in parallel, as well as a minimum number of column-based re-write operations, to the increased delay of RRAM write [21].

This can be summarized in the following mathematical formulation:

$$\min_{S_{sel}} \sum_{k=1}^S S_{sel,k} \cdot (w_{inf} \cdot n_{inf,k} + w_{wr} \cdot n_{wr,k}) \quad (1)$$

which is subject to:

$$\sum_{j=1}^p (a_{k,j} \cdot S_{sel,j}) \geq 1, \quad (2)$$

$S_{sel,k}$ indicates whether the k th column input vector has been selected or not and takes a binary value. Equation (1) has 2 weight values w_{inf} and w_{wr} , with $w_{inf}=w_{wr}=0.5$ and $w_{inf}+w_{wr}=1$, since we aim at limiting both the amount of write operations ($n_{wr,k}$) and overall inference input vector length ($n_{inf,k}$). Equation (1) is responsible for selecting the minimum number of sensitizing sequences, while eq. (2) guarantees that all defects are detected by at least one input in vector S_{sel} . As a result, the binary matrix a , which includes elements $a_{k,j}$ of Equation (2) and encodes the generated fault maps per defect, should include at least one non-zero value.

C. Single-layer Structural Testing

1) *Input Transformation*: Extending the crossbar-level structural tests to single-layer ones is determined by the extra numerical operations used between DNNs. On the input side, floating-point tensors are digitally quantized to match the analog RRAM CIM's target resolution (e.g. 8-bit), with the quantized tensors being applied in a bit-serial fashion rather than via a voltage-based DAC [22]. Those factors significantly differentiate how a potential RRAM single-layer DNN test pattern is transformed and applied to the actual crossbar rows. Also, when performing system-level test development, each layer DNN input produces not a single binary/ternary vector on the crossbar rows, but multiple ones, in particular as many as the input bit resolution. For test development purposes, this implies that a single integer test vector can produce multiple binary ones, thus leading to a potential reduction of the integer-based test patterns in contrast to the structural ones.

2) *Output Transformation*: Next, extending RRAM crossbar readout to a RRAM single-layer DNN, the initial ADC value is fed into an activation function such as ReLU, sigmoid or tanh. Modern DNN architectures also apply tensor operations such as Batch Normalization, pooling and residual connections, which transform all ADC crossbar values, including the potentially defective one [23]. Since we only assume defects on the RRAM crossbar, all subsequent DNN operations can only hinder defect coverage. This makes the need for categorizing DNN fault mask mechanisms imperative for detecting DNN modules and architectures that provide the highest end-to-end DNN testability.

3) *Fault modeling*: Such input-output post-processing could potentially mask initially sensitized defects, by turning the starting shifted value into the nominal non-defective one, by the time it is applied to the rows of the next layer crossbar. Thus, we propose a $FN_L = \langle IN_L/R_L \rangle$ fault notation. IN_L denotes the RRAM single-layer DNN tensor input, in floating point, before any type of quantization. R_L is the RRAM single-layer DNN output difference to the expected, non-defective output tensor, after activation function application and tensor-based digital post-processing. An $R_L = 0$ value implies two cases; one where at least one fault on the FN_{CR} has been successfully sensitized or one where no crossbar fault is generated. In the former case, in-layer masking ($R_L = 0$) could happen due to limited ADC quantization ($R_{ADC} = 0$) within the crossbar input or between ADC output and single layer one. Alternatively, masking could happen due to the operations between the ADC readout and the layer output, leading to $R_{ADC} \neq 0$ and $R_L = 0$. We summarize that the deployment of both FN_{CR} and FN_L gives us the understanding of adequate hardware test patterns for fault sensitization on a crossbar ($S_0, S_1, \dots, S_i$) and layer level (IN_L), circuit-level fault modeling ($F_0, F_1, \dots, F_N$) as well as defect detection (R_{CR}, R_L).

D. System-level Structural Testing

1) *Fault modeling notation*: Full RRAM DNN testing is solely based on the black-box scheme of Fig. 3a, but with

TABLE I: Fault Modeling Table

Symbol	Masking Description	Level
ADC	ADC quantization limit	Circuit
Quant	Input tensor quantization	Layer
ReLU, Sigmoid, Tanh	Activation Function operation	Layer
AvgPool, MaxPool	Pooling operation	Layer
Norm	Batch Normalization	Layer
Zero-MAC/Tile-0-0	Zero-weight multiplication	Layer
Softmax, Clustering, Token	Output layer operation	System

integrating the defect understanding and fault modeling approach of the crossbar and single-layer testing. $FN_{DNN} = \langle IN_{DNN}/L_i/R_{DNN} \rangle$ is used as DNN fault notation, where IN_{DNN} is the DNN input, L_i is the DNN fault notation and R_{DNN} is the difference between defective and nominal DNN based on the DNN output. The existence of a fault masking mechanism is implied via $L_i \neq -$ and zero $R_{DNN} = 0$, meaning that we have a faulty DNN layer whose impact gets sensitized via IN_{DNN} , and then masked via propagation to the subsequent layers, leading to $R_{DNN} = 0$.

2) *Fault masking mechanisms*: Table I encapsulates all potential fault masking mechanisms, which are implied via $L_i \neq -$ and $R_{DNN} = 0$ on the FN_{DNN} . They are separated by the level where the fault mask takes place into circuit/crossbar, intra-layer, and system/DNN levels. On circuit level the predominant factor of masking an initially sensitized fault is the limitation in sensitization by the N-bit deployed ADC. Afterwards, digitized crossbar outputs, as explained in Section III-C2, go through activation functions, such as ReLU, sigmoid, and tanh, as well as pooling and normalization operations. Those numerical transformations all work as potential fault masking points, e.g. via ReLU-based masking of the initial negative defective value, which gets turned to 0. As layer-based fault mask, layer-by-layer propagation can lead to cases where the modified defective tensor gets multiplied with a sparse weight crossbar on one of the subsequent layer. Finally, defective and non-defective propagated values could still lead to the same DNN output, as part of a classification, clustering or generative DNN task.

3) *Input Layer controllability*: As mentioned in section III-A, it is possible to define inference-based test patterns from the input layer in order to test for defects on subsequent ones. For such purpose, we define a 4-layer illustration case based on Fig. 6. Overall, the deeper the layer is located, then the less controllable its input rows are via the DNN image input. Furthermore, different types of images can lead to different results, either in fault sensitization or in no activation of the defect mechanism. Even in the further case, an input test image might not be able to guarantee DNN-level defect detection, in case the output is masked until the DNN output.

Regarding the required layer-by-layer analysis, for the first layer, since the input layer test pattern DNN_{IN} has direct access, we have full controllability to reproduce the targeted

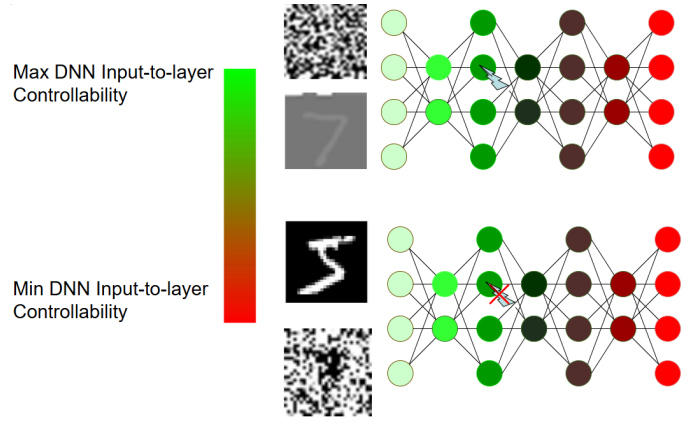


Fig. 6: Input controllability concept

crossbar test vectors of section III-D1 via the first layer inputs.

For layer L_2 , and since we refer to post-training testing with already known DNN weights across all layers, we can similarly determine DNN_{IN} as:

$$DNN_{IN} = L2_{IN} \cdot W_1^+ \quad (3)$$

,where $L2_{IN}$ is the RRAM crossbar target test vector that we want to recreate via DNN_{IN} and W_1^+ is the pseudo-inverse weight matrix of the first layer. As proven in Section V, eq. 3 is not expected to work for the structural test patterns with negative (-1) patterns in the victim input cell, since all deployed DNNs are connected via a ReLU activation function. On the other hand, for negative neighbor cells, they can be generated solely via negative weights and positive inputs, thus making eq. 3 suitable for all other column neighbor cells.

However, for layers L_3 and L_4 , multiple inverse matrix multiplications with previous layer weight tensors does not ensure a correct DNN_{IN} definition due to the existence of non-inversible non-linearities such as the ReLU activation function. As a result, even if we successfully define $L2_{IN} = L3_{IN} \cdot W_2^+$, then all negative values would be masked by the L_2 ReLU A.F.

Due to that inherent non-linearity of DNNs, for layer and test vector combinations where the straight-forward matrix inversion for DNN input determination is infeasible, we define a gradient-based method, via the following loss function:

$$\begin{aligned} \mathcal{L}_1 &= \frac{1}{|M_1|} \sum_{i \in M_1} \max(0, 15.0 - s_i)^2 \\ \mathcal{L}_{-1} &= \frac{1}{|M_{-1}|} \sum_{i \in M_{-1}} \max(0, s_i + 15.0)^2 \\ \mathcal{L}_0 &= \frac{1}{|M_0|} \sum_{i \in M_0} \max(0, |s_i|)^2 \\ \mathcal{L}_{\text{total}} &= \mathcal{L}_1 + \mathcal{L}_{-1} + \mathcal{L}_0 \end{aligned} \quad (4)$$

Bit elements of +1 and -1 are targeted to converge to 4-bit input values of 15 and -15 respectively, since they represent an all 1 matrix of 1111 (either with a positive or a negative sign). Similarly, 0 bit elements on the target vector minimize $\mathcal{L}_0$ when we have convergence on an all zero integer value on

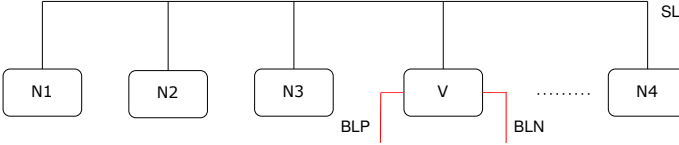


Fig. 7: Input pattern grouping

TABLE II: Device-aware defect modeling [24]

Defect Symbol	Description	Defective Range
OR	Over Reset	$N_{min} \in [0.001 - 0.01] \cdot 10^{26} \text{ m}^{-3}$
ID	Ion Depletion	$N_{min} \in [1.5 - 4.5] \cdot 10^{24} \text{ m}^{-3}$
IUSF	Undefined State Fault	$N_{max} \in [1.5 - 4.5] \cdot 10^{24} \text{ m}^{-3}$
OF	Over Forming	$r_{det} \in [70 - 100] \text{ nm}$
UF	Under Forming	$r_{det} \in [10 - 30] \text{ nm}$

the rows of the crossbar. As a result, after unrolling the '0' integer value, it is ensured that we are going to replicate the targeted 0 bit of the crossbar test pattern generation vector.

IV. APPLICATION OF THE PROPOSED METHODOLOGY

A. Circuit-level simulations

We perform circuit-level defective and non-defective simulations on a 32x2 2T2R RRAM crossbar, with an 8-bit VCO ADC readout circuit [25]. We inject cell-aware resistive defects with 10 defect strengths in the $[100\Omega, 250\text{M}\Omega]$ range. We then apply parallel input test patterns across the 32 rows in groups. The assumed defective cell V is a single group, where as the rest of the $32-1=31$ groups are separated into 4 categories N_1-N_4 . We develop 2 different setups, one where N_1 to N_3 each have 1 unique 2T2R value in the $\{-1,0,1\}$ range and N_4 . Visualization is provided in Fig. 7. To accurately model the RRAM mechanism and its defects, we apply the physics-based JART VCM v1b RRAM model [26]. Defective and non-defective netlists are simulated in Cadence Spectre on a TSMC 40nm process for CMOS and peripheral device simulations. SPICE-level results are analyzed via custom fault analysis and test generation Python Software with SciPy library [27] deployed for solving the formulated ILP problem.

B. RRAM DNN framework development

1) *RRAM Crossbar mapping*: Our analysis is based on post-training emulation, with fixed ternary weights, replicating the latent weight training approach described in [28]. Then all software weight tensors are distributed in 32x32 input tensors, where each of the 1024 elements are having values in the $\{-1,0,1\}$ range. For each DNN layer, we deploy an RRAM tile grid of 32x32 2T2R RRAM tiles, with $\lceil L_{IN}/32 \rceil$ rows and $\lceil L_{OUT}/32 \rceil$, where L_{IN} and L_{OUT} are the input and output layer dimensions. Input rows and weight elements that are not deployed are padded with 0 values to avoid affecting the analog RRAM MAC operation. The outputs of all RRAM tiles are subject to column-based addition and row-based concatenation to produce the final RRAM DNN output. Each of the 32x32 crossbars are fixed at 4-bit input and 8-bit output resolution. Other operations such as Normalization, quantization, Activation Functions and Pooling are done on a purely software level, emulating the conventional operation split in analog RRAM-based CIM DNNs [29].

2) *DNN Fault modeling*: System-level fault modeling is the equivalent of fault mask analysis tracing based on Table I. We start by mapping each crossbar-level sensitized faults into an integer-based ADC offset $ADC_{off} \in [-255, 255]$ range from the defective column. Next, we perform DNN inference on the RRAM tile grid with the injected SPICE-aware ADC offset and for each subsequent operation of I we check whether or not DNN simulations with and without the injected offset lead to the same output value. If that is the case, then we flag the case as masked and report the masking mechanism and the layer of occurrence according to the FN_{DNN} notation.

V. RESULTS OF THE PROPOSED METHODOLOGY

A. Crossbar testing results

1) *Fault map analysis*: The crossbar-level results start by validating the FN_{CR} parts of the notation, i.e. the sensitization inputs, the RRAM-based produced fault, as well as the ADC-based readout difference. This is summarized in the fault maps of Fig. 8. The fault maps are split up in a grid style, where columns showcase the different column neighbor scheme (from 0 neutral 2T2R cells to all neutrals) and rows indicate the initial write operation - or lack of it e.g. in the case of ADC defect simulation - performed on the assumed victim cell. Next, the coloring of the fault maps showcases the numerical difference of the victim cell V on before and after the input operation. For instance, Fig. 8b includes cases where the 0w1 operation fails, thus leading to a negative (-1) cell state difference between expected positive state (1) and the actual one (0), whereas for the -1w0 input category, we notice similar write failure operation for high resistive strength open defective values, notated with a (+1) orange value. Apart from the fault modeling inclusion, fault maps also incorporate defect case detection with bold color, in contrast to the staid one for undetected cases. Finally, within each tile grid, the row inputs that belong in the same write and neighbor category have been sorted from the most negative to the most positive one, based on the scheme of Fig. 7. The columns of each tile grid sweep the defect strength to a 100Ω - $250\text{M}\Omega$ range.

2) *Defect type fault map comparison*: Fig. 8 gives us preliminary insights for defect categories. Fig. 8a is the fault map result for RRAM defects of over-reset and ion-depletion. Both of those defects are modeled via an increasing sweep in the minimum nominal oxide concentration $N_{discmin}$, based on Table II. We notice that fault sensitization and subsequent defect detection come solely from input patterns in the 0w-1 and -1w-1 categories, which are the equivalent of 1w0 for an 1T1R cell. In addition to the limited test pattern selection, we also have very low defect coverage due to the minor effect of the reset-based defects to the overall accumulated SL current in analog RRAM CIM crossbars. On the other hand, resistive intra-cell and ADC defects of Figs 8b and 8c are much easier to detect for more defect strength cases, with their main difference regarding fault modeling. For instance, in Fig 8b, we have multiple RRAM flips of +1 and -1, especially in input categories 0w1, 0w-1, -1w0. Actually, in the latter case, the -1w0 operation is actually enough to make the low resistance

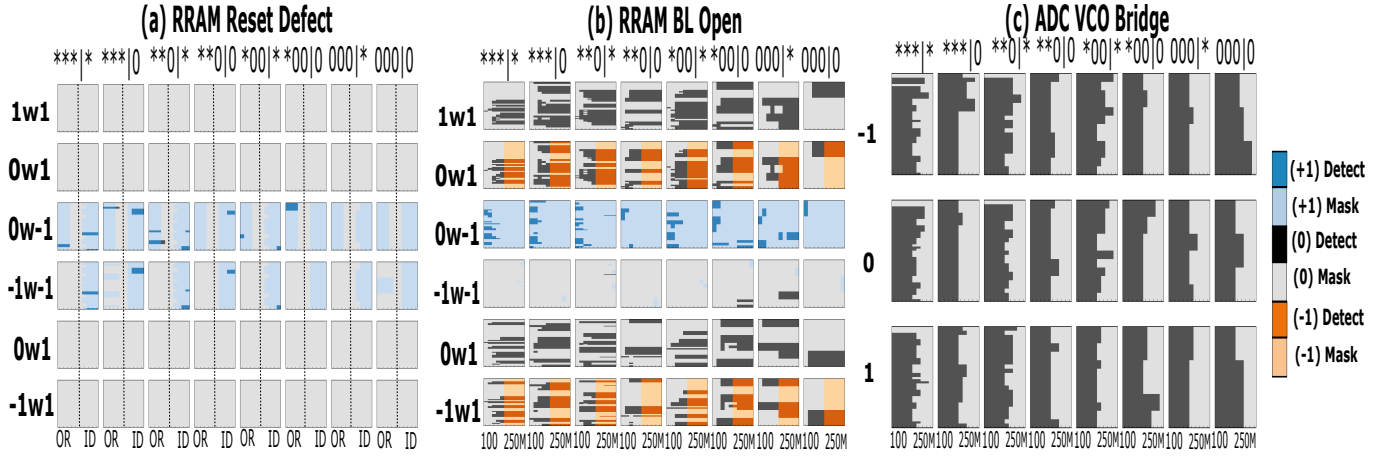


Fig. 8: Fault maps for 3 different types of defects (a) OverReset and Ion Depletion RRAM Defects (b) 2T2R Open (c) ADC Oscillator Bridge

cases detectable and thus guarantee full defect coverage for such defect occasion. In contrast, ADC defects, since not applied in the RRAM cells, do not sensitize any faults, but still lead to different ADC output code for most defect strengths.

3) *Balanced Test Pattern Scheme Validation:* We evaluate 3 different input setups based on Fig. 7. Next, the 32x2 simulation setup is divided further into one with $N_1=N_2=N_3=1$, one with $N_1=N_2=N_3=5$ and another one with $N_1=N_2=N_3=7$ in order to quantify the effect of an increased and decreased amount of N_4 bulk elements respectively. As shown in Fig. 9, both unbalanced and balanced fault maps are comparable in terms of sensitized RRAM 2T2R faults and fault coverage. However, in the $**0|0$ neighbor category of Fig. 9b, we have marked as green certain patterns which cover more defect strength values. It should be noted that this defect coverage improvement is not related to any fault sensitization, thus validating the effect of the balanced neighboring scheme. These results validate the empirical estimation that having a large bulk of cells limits the detection range of the parallel column operations, a problem more severe for larger arrays.

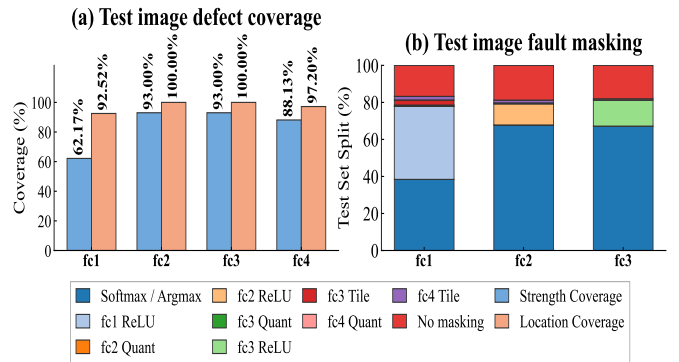
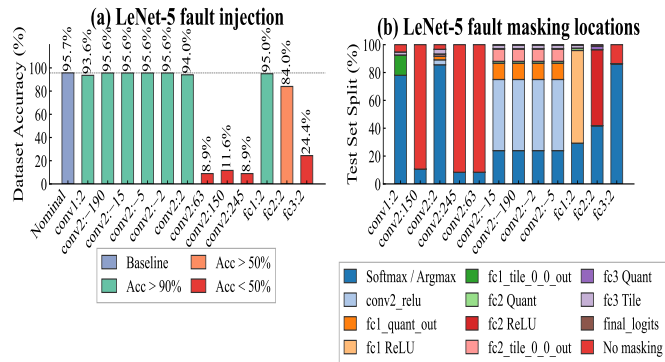
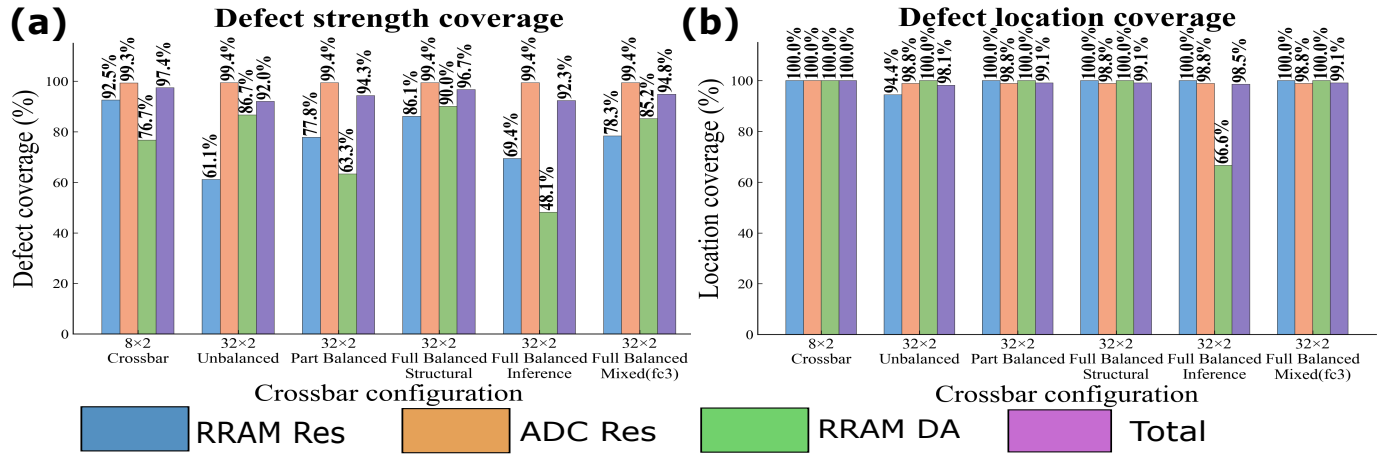
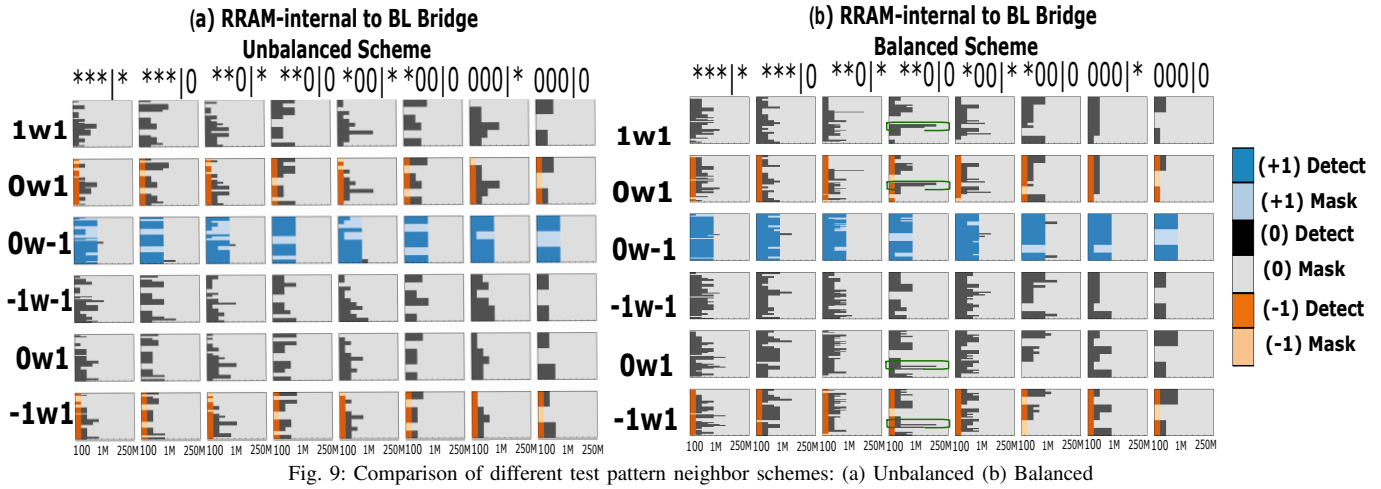
Similar summary results are shown in Fig. 10. For completeness reasons, we have also included the 8x2 crossbar results from [30, 31]. As expected, the 8x2 setup, which incorporates just 4 bulk cells offers the highest defect coverage across all defect types (VCO ADC, 2T2R resistive and RRAM device defects). Between the 2 proposed 32x2 setups, the balanced scheme provides a 15% improvement in RRAM resistive defect, which subsequently leads to an overall improvement from 92% to 94.3%. Similarly, unique defect location detection is again in favor of the balanced scheme, due to the inability of the unbalanced neighbor scheme to detect WL defects.

B. DNN Hardware-aware Fault analysis

1) *Functional Fault Analysis:* Based on the FN_{CR} and FN_L notations we quantify defect detection in proportion to the 8-bit ADC output code difference between golden and defective case. Initially, we perform functional fault analysis on LeNet5 - via ADC offset injection and fault masking - with results shown in Figs. 11a and 11b. Instead of including all

generated offsets from the SPICE defect simulation data, we have selected the local maxima ADC offsets for DNN-based fault injection. For the combination of LeNet5 and the MNIST dataset, we observe that low value addition or subtraction to the first column ADC of the first 2 layers results either in no task accuracy drop-off (fc1:2) or very minor one (conv2:-2 and conv2:2). On the contrary, we observe a large drop-off in accuracy based on the sign of the ADC offset fault injection, as large negative offsets are mostly masked by ReLU and thus not affecting workload accuracy significantly. Large positive offsets in conv2 seem to be severe with all non-negative cases hovering around 10%. Finally, large conv2 ADC offsets, such as +63, +150 and +245, do not follow a proportional decrease in accuracy for ADC offset increase, since conv2:150 leads to higher MNIST accuracy than conv2:63. That is due to the fact that the deployed VCO-ADC is based on an asynchronous counter, which inherently resets to 0 after reaching maximum 8-bit value (255). Regarding fault masking, we notice that the predominant factors are ReLU activation function and as well as the output layer softmax. For the severely affected conv2, we also have the presence of zero-mac (fc1-tile-0-0-out) and quantization effects to DNN fault mask.

2) *Structural fault analysis:* We perform the same fault injection & masking coupled analysis, but now on a 4-layer MLP and not with an actual dataset but on the proposed structural images of Fig. 13, with results shown in Figs. 12a and 12b. Starting from Fig. 12a, which depicts end-to-end defect coverage with output capturing done at the DNN output instead of DNN accuracy as in Fig. 11a. Injected defects in fc1 are the hardest to detect, since propagation until the output of fc4 masks $\approx 40\%$ of all defect cases. On the other hand, layer fc2 and fc3 are tested in a hybrid way, since in Fig. 13, not all structural test patterns are controllable via test input images targeting fc2 & fc3. Finally, regarding fc4, which cannot deploy any end-to-end test images, it has the highest defect coverage, both due to its location being closest to DNN output, and due to the deployment of structural test patterns, which are designed for maximum defect detection. Similarly, to 11b, output softmax and inter-layer ReLU are the



dominant fault masking factors for most test image cases, with the existence of quantization and zero-mac operation cases being much more limited in quantity in contrast to the more fault sensitive LeNet5.

C. Input controllability results

1) *Synthetic Image Generation*: The aim of input-based controllability is to provide a small subset of images which

is going to reproduce the structural test patterns of Section V-A during inference operation, removing the need of layer-by-layer access and write operations and reducing overall test time. The overall results for all 3 image generation methods, i.e. inverse matrix multiplication, dataset search and gradient optimization, are provided in Fig. 13. The pattern IDs (Pattern 1 to Pattern 11) follow exactly the same indexing as the

TABLE III: Structural test patterns generated for the fully balanced 32x2 crossbar.

Pattern	W_{in}	V_{in}	Neighbors	Bulk	Init State	Type
1	-1	0	PNP	N	—	RRAM
2	-1	1	PON	0	—	RRAM
3	1	-1	PN0	0	—	RRAM
4	1	-1	NPP	0	—	RRAM
5	1	1	PPN	0	—	RRAM
6	-1	0	PPP	P	—	ADC
7	1	1	NNN	N	—	ADC
8	0	0	P00	0	—	ADC
9	1	0	PP0	N	HRS	ADC
10	0	-1	PPP	0	HRS	ADC
11	0	0	000	P	HRS	ADC

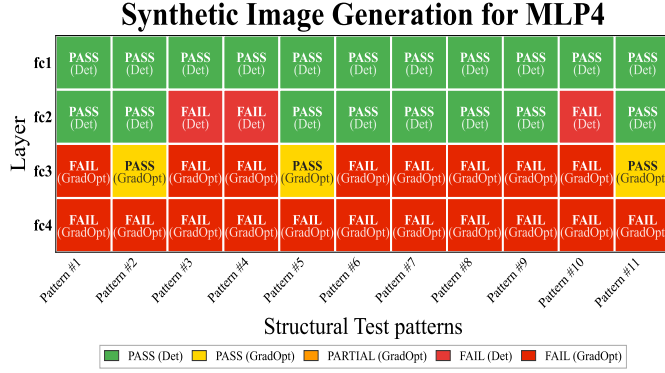


Fig. 13: Input controllability results for all ternarized structural test patterns

provided single-layer test patterns of Table III. Here, matrix inversion covers all 11 ternary test vectors for fc1 and fc2, whereas for fc3 and fc4 we rely solely on gradient optimization which generates the wanted input image $3/9 = 33\%$ of the time. This makes fc3 and fc4 ternary inputs hard-to-control for the majority of structural test patterns. Next, Fig. 14 offers specific synthetic images per targeted layer. We notice that for fc1 and fc3 layers, images seem less random than for fc2 and fc4, since the 2 further cases are based on first-layer matrix inversion and gradient optimization respectively. Specifically, for layer fc3, the generated input image is basically a noisy '0' symbol. That comes as a result of the pre-seed selection of our algorithm, according to which we start the gradient optimization from the best MNIST-based or deterministic image. On the other hand, performing either pipeline 2 layers after results in a significantly more random input image.

2) *Structural vs Image-based Comparison:* Based on the above, we can estimate the test cost reduction of our synthetic image approach. For the structural test setup, we include the average ADC-based column read latency of $500ns$ and write initialization operation of $10us$, as those values were deployed on the custom SPICE defect simulation setup. Next, the black-box DNN setup only applies the test images in an end-to-end fashion, without any controllability or observability between layers. Finally, for the hybrid approach, we apply test images based on the controllability results of 13, and for the non-controllable ones apply the structural test vectors of Table III. For RRAM DNN inference, a $2us$ per-layer inference time

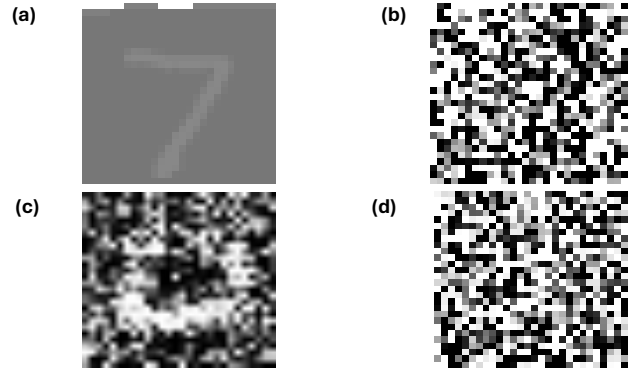


Fig. 14: Generated synthetic image examples for (a) FC1 inverse matrix generation (PASS) (b) FC2 inverse matrix generation (PASS) (c) FC3 GradOpt Generation (PASS) (d) FC4 GradOpt (FAIL)

TABLE IV: Expected test time and defect coverage comparison between all 3 generated methods

Test Type	Time(s)	Strength Coverage	Location Coverage
Structural only	9.45ms	96.7%	99.1%
Black box	2.7ms	85%	97.4%
Hybrid	5.55ms	94.46%	98.95%

(4 ADC conversions of $500ns$ per layer) is expected. This leads to a $4 \cdot 2us = 8us$ standard latency per test image for the black-box testing, but a linearly increasing layer latency of $d_{fc,i} = i \cdot 2us$ for the test images of the hybrid approach, where i is the targeted layer ($i \in [1,4]$ for the 4 layer MLP examined). In all 3 setups, we do not include the latency added due to digital post-processing, since that is expected to be similar for all test setups. Also, for the purely structural method, we include the conversion of each of the 11 initial test pattern generation vectors into a set of 225 binary shifted arrays, out of the $32 \cdot 11 = 352$ total potential test patterns, with the rest of $384 - 225 = 127$ binary crossbar vectors being duplicates.

That leads to the results of Table IV where we can reduce by ≈ 2 and ≈ 4 times test latency with the hybrid and black-box methods respectively, which includes both inference images and structural test patterns. That is, since most of the patterns of both approaches for fc1 & fc2 layers are generated via synthetic images, based on the results of 6. Moreover, the slight drop-off in defect detection between the fully structural and the inference based ones is due to the fact that, as shown in Fig. 10, the inference-based fully balanced test pattern generation scheme leads to a significant reduction of the RRAM defect coverage from $\approx 86\%$ to $\approx 66\%$ and thus the overall accuracy to 92.3% from the maximum 96.7% . Also, the reported 94.46% of the hybrid accuracy is based on the defect coverage results of Fig. 10a, with a 94.20% coverage included for fc2. Such a tradeoff affects mostly the test patterns of layers fc1 to fc3 which come with a 'PASS' tag in Fig. 6, thus are being replaced with equivalent high-speed but lower defect coverage test images. From the table, we can safely assume that, unlike the end-to-end testing, the hybrid approach can keep an adequately high system-level defect strength & coverage, while still significantly reducing test time overhead.

TABLE V: Test time & defect coverage comparison for structural and functional RRAM tests

Test	Defect Modeling	Test Inputs	Final Coverage
Backdoored Checksums[10]	SA0/SA1	10000	92%
Alternative Testing [32]	Variations	300	95.4%
RRAM BIST [33]	SA0/SA1	10000	95.47%
RRAMedy [34]	SA0/SA1, Disturbance	9553	93%
This work	Cell+Device-aware	900	94.46%

VI. DISCUSSION

Comparison to other works: Table V provides a head-to-head comparison of our test approach to other structural and functional ones found in literature. For our method, we select the hybrid structural-inference case which comes with a great balance of reduced test time and still maximum defect coverage as the exhaustive and time consuming structural test case of Table IV. As we notice, our work has very comparable average defect coverage to SOTA, with only Alternative Test [32] coming at a lower amount of deployed inputs and better coverage. However, the defect realism of [10, 33, 34] is limited to just stuck-at faults whereas the circuit simulations of [32] mainly target permanent faults and conductance variation, without exhaustive analysis of the cell and peripheral defect space. Moreover, by selecting the structural test approach, we can increase final defect coverage to the nominal 96.7% value of write+read crossbar testing.

Test optimizations: We demonstrated that the detection capabilities depends strongly on the possibility to control the inputs and observe the outputs of a layer. These insights can be used to develop quicker tests or have more defect coverage. For example, when we add additional control so that the inputs to fc3 & fc4 can be set, the defect coverage increases, as better test patterns can be applied. Similarly, when we are able to observe the outputs of fc1 & fc2 directly, we do not need as many patterns to be able to propagate them to the output of the complete DNN, which reduces test time.

VII. CONCLUSION

This work proposes a full-stack structural testing framework for analog RRAM CIM accelerators. We define multi-level fault models, compare test pattern generation for cell and peripheral defects, and analyze their impact on DNN functional safety and fault masking. Finally, we evaluate the trade-offs between structural and synthetic inference testing regarding defect coverage and time overhead.

Future aspects include:

Inclusion of more advanced DNN models: Our full-stack test approach included fully connected layers and convolutional layers with other structures like LSTM and attention-based mechanisms having to be explored for full validation.

Multi-level analysis: Our circuit setup was based on ternary RRAM cells, with the inclusion of multi-level programming states and other memory technologies being necessary for understanding testing of memristive CIM AI.

Design of DFT circuits: As proven in the paper, defect coverage ceiling via adequate fault sensitization exists solely on the crossbar-level. DFT input/output schemes could be deployed in order to sense HtD RRAM defects more reliably.

REFERENCES

- [1] J. T. Maurer et al., "A survey on computing-in-memory (cim) and emerging nonvolatile memory (nvm) simulators," *Chips*, 2025.
- [2] A. El Arrassi et al., "Dream-cim: A digital sram-based cim accelerator for energy- and area-efficient edge ai," *IEEE TCAS-AI*, 2025.
- [3] W. Wan et al., "A compute-in-memory chip based on resistive random-access memory," *Nature*, vol. 608, pp. 504–512, 2022.
- [4] M. Fieback et al., "Defects, fault modeling, and test development framework for rrams," *J. Emerg. Technol. Comput. Syst.*, Apr. 2022.
- [5] W. Banerjee et al., "Engineering of defects in resistive random access memory devices," *JAP*, p. 051 101, 2020.
- [6] T. S. Copetti et al., "A dft strategy for guaranteeing ReRAM's quality after manufacturing," *JET*, 2024.
- [7] Z. Lin et al., "Analysis and design of memory testing algorithm for computing-in-memory using mbist," *JET-CS*, Jan. 2026.
- [8] M. Liu et al., "Online fault detection in rram-based computing systems for inferencing," *IEEE TVLSI*, 2022.
- [9] C.-W. Hou et al., "Fault modeling and testing of memristor-based spiking neural networks," in *2022 ITC*, 2022, pp. 92–99.
- [10] C.-Y. Chen et al., "On-line functional testing of memristor-mapped dnn using backdoored checksums," in *ITC*, IEEE, 2021.
- [11] C.-Y. Chen et al., "Efficient identification of critical faults in memristor crossbars for deep neural networks," in *DATE*, IEEE, 2021.
- [12] L. Xia et al., "Rescuing memristor-based neuromorphic design with high defect rate," in *DAC*, 2017.
- [13] N. Ahmed et al., "Sipt: Signature-based predictive testing of rram crossbar arrays for deep neural networks," *IEEE TC*, 2025.
- [14] A. Mazaheri et al., "Reliable binarized neural networks on unreliable beyond von-neumann architecture," *IEEE TCAD-ICS*, 2022.
- [15] X. Peng et al., "Dnn-neurosim: An end-to-end benchmarking framework for compute-in-memory accelerators with versatile device technologies," in *2019 IEEE IEDM*, 2019.
- [16] X. Khafa et al., "Sram periphery testing using the cell-aware test methodology," *IEEE TCAD-ICS*, 2025.
- [17] M. Fieback et al., "Device-aware test: A new test approach towards dppb level," in *ITC*, 2019.
- [18] A. Gebregiorgis et al., "Tutorial on memristor-based computing for smart edge applications," *M-MDCS*, 2023.
- [19] Q. Liu et al., "33.2 a fully integrated analog rram based 78.4tops/w compute-in-memory chip with fully parallel mac computing," in *ISSCC*, 2020, pp. 500–502.
- [20] S. Hamdioui et al., "An Experimental Analysis of Spot Defects in SRAMs: Realistic Fault Models and Tests," in *ATS*, 2000.
- [21] S. K. Dubey et al., "Architecture of resistive ram with write driver," *Solid State Electronics Letters*, vol. 2, pp. 10–22, 2020.
- [22] S. Diware et al., "Unbalanced bit-slicing scheme for accurate memristor-based neural network architecture," in *2021 IEEE AICAS*.
- [23] M. Vakalopoulou et al., "Deep learning: Basics and convolutional neural networks (cnns)," in *ML for Brain Disorders*. 2023, pp. 77–115.
- [24] H. Xun et al., "Robust design-for-testability scheme for conventional and unique defects in rrams," in *ITC*, 2024.
- [25] M. Mayahinia et al., "A voltage-controlled, oscillation-based adc design for computation-in-memory architectures using emerging rrams," *J. Emerg. Technol. Comput. Syst.*, Mar. 2022.
- [26] F. Cüppers et al., "Exploiting the Switching Dynamics of HfO₂-based ReRAM Devices for Reliable Analog Memristive Behavior," *APL*,
- [27] P. Virtanen et al., "Scipy 1.0: Fundamental algorithms for scientific computing in python," *Nature methods*, vol. 17, no. 3, 2020.
- [28] F. Li et al., "Ternary weight networks," *CoRR*, vol. abs/1605.04711, 2016. arXiv: 1605.04711.
- [29] J. Büchel et al., "Analog foundation models," *ArXiv*, vol. abs/2505.09663, 2025.
- [30] E. A. Serlis et al., "Structural testing of a rram-based ai accelerator core," in *2025 IEEE ETS*.
- [31] E. A. Serlis et al., "Combined array and adc structural test for rram-based multiply-and-accumulate circuits," in *2025 ITC*, 2025.
- [32] K. Ma et al., "Efficient low cost alternative testing of analog crossbar arrays for deep neural networks," in *2022 IEEE ITC*, 2022.
- [33] A. K. Mishra et al., "Built-in functional testing of analog in-memory accelerators for deep neural networks," *Electronics*, 2022.
- [34] W. Li et al., "Rramedy: Protecting rram-based neural network from permanent and soft faults during its lifetime," in *ICCD*, 2019.